

SEI SERIES IN SOFTWARE ENGINEERING

Software Architecture in Practice

FOURTH EDITION

Len Bass

Paul Clements

Rick Kazman



Software Architecture in Practice

Fourth Edition

The SEI Series in Software Engineering



Software Engineering Institute

Carnegie Mellon University



Visit informit.com/sei for a complete list of available publications.

The SEI Series in Software Engineering is a collaborative undertaking of the Carnegie Mellon Software Engineering Institute (SEI) and Addison-Wesley to develop and publish books on software engineering and related topics. The common goal of the SEI and Addison-Wesley is to provide the most current information on these topics in a form that is easily usable by practitioners and students.

Titles in the series describe frameworks, tools, methods, and technologies designed to help organizations, teams, and individuals improve their technical or management capabilities. Some books describe processes and practices for developing higher-quality software, acquiring programs for complex systems, or delivering services more effectively. Other books focus on software and system architecture and product-line development. Still others, from the SEI's CERT Program, describe technologies and practices needed to manage software and network security risk. These and all titles in the series address critical problems in software engineering for which practical solutions are available.



Make sure to connect with us!
informit.com/socialconnect

Software Architecture in Practice

Fourth Edition

Len Bass

Paul Clements

Rick Kazman

◆ Addison-Wesley

Boston • Columbus • New York • San Francisco • Amsterdam • Cape Town

Dubai • London • Madrid • Milan • Munich • Paris • Montreal • Toronto • Delhi • Mexico City

São Paulo • Sydney • Hong Kong • Seoul • Singapore • Taipei • Tokyo



Software Engineering Institute

Carnegie Mellon

The SEI Series in Software Engineering

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and the publisher was aware of a trademark claim, the designations have been printed with initial capital letters or in all capitals.

CMM, CMMI, Capability Maturity Model, Capability Maturity Modeling, Carnegie Mellon, CERT, and CERT Coordination Center are registered in the U.S. Patent and Trademark Office by Carnegie Mellon University.

ATAM; Architecture Tradeoff Analysis Method; CMM Integration; COTS Usage-Risk Evaluation; CURE; EPIC; Evolutionary Process for Integrating COTS Based Systems; Framework for Software Product Line Practice; IDEAL; Interim Profile; OAR; OCTAVE; Operationally Critical Threat, Asset, and Vulnerability Evaluation; Options Analysis for Reengineering; Personal Software Process; PLTP; Product Line Technical Probe; PSP; SCAMPI; SCAMPI Lead Appraiser; SCAMPI Lead Assessor; SCE; SEI; SEPG; Team Software Process; and TSP are service marks of Carnegie Mellon University.

Special permission to reproduce portions of works copyright by Carnegie Mellon University, as listed on page 437, is granted by the Software Engineering Institute.

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and the publisher was aware of a trademark claim, the designations have been printed with initial capital letters or in all capitals.

The authors and publisher have taken care in the preparation of this book, but make no expressed or implied warranty of any kind and assume no responsibility for errors or omissions. No liability is assumed for incidental or consequential damages in connection with or arising out of the use of the information or programs contained herein.

For information about buying this title in bulk quantities, or for special sales opportunities (which may include electronic versions; custom cover designs; and content particular to your business, training goals, marketing focus, or branding interests), please contact our corporate sales department at corpsales@pearsoned.com or (800) 382-3419.

For government sales inquiries, please contact governmentsales@pearsoned.com.

For questions about sales outside the U.S., please contact intlcs@pearson.com.

Visit us on the Web: informit.com/aw

Library of Congress Control Number: 2021934450

Copyright © 2022 Pearson Education, Inc.

Cover image: [Zhernosek_FFMstudio.com/Shutterstock](https://www.shutterstock.com/image-vector/zhernosek_FFMstudio.com/Shutterstock)

Hand/input icon: [In-Finity/Shutterstock](https://www.shutterstock.com/image-vector/in-finity/Shutterstock)

Figure 1.1: [GraphicsRF.com/Shutterstock](https://www.shutterstock.com/image-vector/graphicsrf.com/Shutterstock)

Figure 15.2: [Shutterstock Vector/Shutterstock](https://www.shutterstock.com/image-vector/shutterstock-vector/Shutterstock)

Figure 17.1: [Oleksiy Mark/Shutterstock](https://www.shutterstock.com/image-vector/oleksiy-mark/Shutterstock)

Figure 17.2, cloud icon: [luckyguy/123RF](https://www.shutterstock.com/image-vector/luckyguy/123RF)

Figures 17.2, 17.4, and 17.5 computer icons: [Dacian G/Shutterstock](https://www.shutterstock.com/image-vector/dacian-g/Shutterstock)

All rights reserved. This publication is protected by copyright, and permission must be obtained from the publisher prior to any prohibited reproduction, storage in a retrieval system, or transmission in any form or by any means, electronic, mechanical, photocopying, recording, or likewise. For information regarding permissions, request forms and the appropriate contacts within the Pearson Education Global Rights & Permissions Department, please visit www.pearson.com/permissions/.

ISBN-13: 978-0-13-688609-9

ISBN-10: 0-13-688609-4

ScoutAutomatedPrintCode

Contents

Preface *xv*

Acknowledgments *xvii*

PART I INTRODUCTION 1

CHAPTER 1 What Is Software Architecture? 1

- 1.1 What Software Architecture Is and What It Isn't 2
- 1.2 Architectural Structures and Views 5
- 1.3 What Makes a “Good” Architecture? 19
- 1.4 Summary 21
- 1.5 For Further Reading 21
- 1.6 Discussion Questions 22

CHAPTER 2 Why Is Software Architecture Important? 25

- 2.1 Inhibiting or Enabling a System's Quality Attributes 26
- 2.2 Reasoning about and Managing Change 27
- 2.3 Predicting System Qualities 28
- 2.4 Communication among Stakeholders 28
- 2.5 Early Design Decisions 31
- 2.6 Constraints on Implementation 31
- 2.7 Influences on Organizational Structure 32
- 2.8 Enabling Incremental Development 33
- 2.9 Cost and Schedule Estimates 33
- 2.10 Transferable, Reusable Model 34
- 2.11 Architecture Allows Incorporation of Independently Developed Elements 34

2.12 Restricting the Vocabulary of Design Alternatives	35
2.13 A Basis for Training	36
2.14 Summary	36
2.15 For Further Reading	37
2.16 Discussion Questions	37

PART II QUALITY ATTRIBUTES 39

CHAPTER 3 Understanding Quality Attributes 39

3.1 Functionality	40
3.2 Quality Attribute Considerations	41
3.3 Specifying Quality Attribute Requirements: Quality Attribute Scenarios	42
3.4 Achieving Quality Attributes through Architectural Patterns and Tactics	45
3.5 Designing with Tactics	46
3.6 Analyzing Quality Attribute Design Decisions: Tactics-Based Questionnaires	48
3.7 Summary	49
3.8 For Further Reading	49
3.9 Discussion Questions	50

CHAPTER 4 Availability 51

4.1 Availability General Scenario	53
4.2 Tactics for Availability	55
4.3 Tactics-Based Questionnaire for Availability	62
4.4 Patterns for Availability	66
4.5 For Further Reading	68
4.6 Discussion Questions	69

CHAPTER 5 Deployability 71

5.1 Continuous Deployment	72
5.2 Deployability	75
5.3 Deployability General Scenario	76
5.4 Tactics for Deployability	78

5.5	Tactics-Based Questionnaire for Deployability	80
5.6	Patterns for Deployability	81
5.7	For Further Reading	87
5.8	Discussion Questions	87
CHAPTER 6	Energy Efficiency	89
6.1	Energy Efficiency General Scenario	90
6.2	Tactics for Energy Efficiency	92
6.3	Tactics-Based Questionnaire for Energy Efficiency	95
6.4	Patterns	97
6.5	For Further Reading	98
6.6	Discussion Questions	99
CHAPTER 7	Integrability	101
7.1	Evaluating the Integrability of an Architecture	102
7.2	General Scenario for Integrability	104
7.3	Integrability Tactics	105
7.4	Tactics-Based Questionnaire for Integrability	110
7.5	Patterns	112
7.6	For Further Reading	114
7.7	Discussion Questions	115
CHAPTER 8	Modifiability	117
8.1	Modifiability General Scenario	120
8.2	Tactics for Modifiability	121
8.3	Tactics-Based Questionnaire for Modifiability	125
8.4	Patterns	126
8.5	For Further Reading	130
8.6	Discussion Questions	131
CHAPTER 9	Performance	133
9.1	Performance General Scenario	134
9.2	Tactics for Performance	137
9.3	Tactics-Based Questionnaire for Performance	145
9.4	Patterns for Performance	146

9.5	For Further Reading	149
9.6	Discussion Questions	150

CHAPTER 10 Safety 151

10.1	Safety General Scenario	154
10.2	Tactics for Safety	156
10.3	Tactics-Based Questionnaire for Safety	160
10.4	Patterns for Safety	163
10.5	For Further Reading	165
10.6	Discussion Questions	166

CHAPTER 11 Security 169

11.1	Security General Scenario	170
11.2	Tactics for Security	172
11.3	Tactics-Based Questionnaire for Security	176
11.4	Patterns for Security	179
11.5	For Further Reading	180
11.6	Discussion Questions	180

CHAPTER 12 Testability 183

12.1	Testability General Scenario	186
12.2	Tactics for Testability	187
12.3	Tactics-Based Questionnaire for Testability	192
12.4	Patterns for Testability	192
12.5	For Further Reading	194
12.6	Discussion Questions	195

CHAPTER 13 Usability 197

13.1	Usability General Scenario	198
13.2	Tactics for Usability	200
13.3	Tactics-Based Questionnaire for Usability	202
13.4	Patterns for Usability	203
13.5	For Further Reading	205
13.6	Discussion Questions	205

CHAPTER 14	Working with Other Quality Attributes	207
14.1	Other Kinds of Quality Attributes	207
14.2	Using Standard Lists of Quality Attributes—Or Not	209
14.3	Dealing with “X-Ability”: Bringing a New QA into the Fold	212
14.4	For Further Reading	215
14.5	Discussion Questions	215
PART III	ARCHITECTURAL SOLUTIONS	217
CHAPTER 15	Software Interfaces	217
15.1	Interface Concepts	218
15.2	Designing an Interface	222
15.3	Documenting the Interface	228
15.4	Summary	230
15.5	For Further Reading	230
15.6	Discussion Questions	231
CHAPTER 16	Virtualization	233
16.1	Shared Resources	234
16.2	Virtual Machines	235
16.3	VM Images	238
16.4	Containers	239
16.5	Containers and VMs	241
16.6	Container Portability	242
16.7	Pods	242
16.8	Serverless Architecture	243
16.9	Summary	244
16.10	For Further Reading	245
16.11	Discussion Questions	245
CHAPTER 17	The Cloud and Distributed Computing	247
17.1	Cloud Basics	248
17.2	Failure in the Cloud	251

17.3	Using Multiple Instances to Improve Performance and Availability	253
17.4	Summary	261
17.5	For Further Reading	262
17.6	Discussion Questions	262

CHAPTER 18 Mobile Systems 263

18.1	Energy	264
18.2	Network Connectivity	266
18.3	Sensors and Actuators	267
18.4	Resources	268
18.5	Life Cycle	270
18.6	Summary	273
18.7	For Further Reading	274
18.8	Discussion Questions	275

PART IV SCALABLE ARCHITECTURE PRACTICES 277

CHAPTER 19 Architecturally Significant Requirements 277

19.1	Gathering ASRs from Requirements Documents	278
19.2	Gathering ASRs by Interviewing Stakeholders	279
19.3	Gathering ASRs by Understanding the Business Goals	282
19.4	Capturing ASRs in a Utility Tree	284
19.5	Change Happens	286
19.6	Summary	286
19.7	For Further Reading	287
19.8	Discussion Questions	287

CHAPTER 20 Designing an Architecture 289

20.1	Attribute-Driven Design	289
20.2	The Steps of ADD	292
20.3	More on ADD Step 4: Choose One or More Design Concepts	295
20.4	More on ADD Step 5: Producing Structures	298

20.5	More on ADD Step 6: Creating Preliminary Documentation during the Design	301
20.6	More on ADD Step 7: Perform Analysis of the Current Design and Review the Iteration Goal and Achievement of the Design Purpose	304
20.7	Summary	306
20.8	For Further Reading	306
20.9	Discussion Questions	307
CHAPTER 21	Evaluating an Architecture	309
21.1	Evaluation as a Risk Reduction Activity	309
21.2	What Are the Key Evaluation Activities?	310
21.3	Who Can Perform the Evaluation?	311
21.4	Contextual Factors	312
21.5	The Architecture Tradeoff Analysis Method	313
21.6	Lightweight Architecture Evaluation	324
21.7	Summary	326
21.8	For Further Reading	327
21.9	Discussion Questions	327
CHAPTER 22	Documenting an Architecture	329
22.1	Uses and Audiences for Architecture Documentation	330
22.2	Notations	331
22.3	Views	332
22.4	Combining Views	339
22.5	Documenting Behavior	340
22.6	Beyond Views	345
22.7	Documenting the Rationale	346
22.8	Architecture Stakeholders	347
22.9	Practical Considerations	350
22.10	Summary	353
22.11	For Further Reading	353
22.12	Discussion Questions	354

CHAPTER 23	Managing Architecture Debt	355
23.1	Determining Whether You Have an Architecture Debt Problem	356
23.2	Discovering Hotspots	358
23.3	Example	362
23.4	Automation	363
23.5	Summary	364
23.6	For Further Reading	364
23.7	Discussion Questions	365
PART V	ARCHITECTURE AND THE ORGANIZATION	367
CHAPTER 24	The Role of Architects in Projects	367
24.1	The Architect and the Project Manager	367
24.2	Incremental Architecture and Stakeholders	369
24.3	Architecture and Agile Development	370
24.4	Architecture and Distributed Development	373
24.5	Summary	376
24.6	For Further Reading	376
24.7	Discussion Questions	377
CHAPTER 25	Architecture Competence	379
25.1	Competence of Individuals: Duties, Skills, and Knowledge of Architects	379
25.2	Competence of a Software Architecture Organization	386
25.3	Become a Better Architect	387
25.4	Summary	388
25.5	For Further Reading	388
25.6	Discussion Questions	389

PART VI	CONCLUSIONS	391
CHAPTER 26	A Glimpse of the Future: Quantum Computing	391
26.1	Single Qubit	392
26.2	Quantum Teleportation	394
26.3	Quantum Computing and Encryption	394
26.4	Other Algorithms	395
26.5	Potential Applications	396
26.6	Final Thoughts	397
26.7	For Further Reading	398
	<i>References</i>	399
	<i>About the Authors</i>	415
	<i>Index</i>	417

This page intentionally left blank

Preface

When we set out to write the fourth edition of *Software Architecture in Practice*, our first question to ourselves was: Does architecture still matter? With the rise of cloud infrastructures, microservices, frameworks, and reference architectures for every conceivable domain and quality attribute, one might think that architectural knowledge is hardly needed anymore. All the architect of today needs to do is select from the rich array of tools and infrastructure alternatives out there, instantiate and configure them, and voila! An architecture.

We were (and are) pretty sure this is not true. Admittedly, we are somewhat biased. So we spoke to some of our colleagues—working architects in the healthcare and automotive domains, in social media and aviation, in defense and finance and e-commerce—none of whom can afford to let dogmatic bias rule them. What we heard confirmed our belief—that architecture is just as relevant today as it was more than 20 years ago, when we wrote the first edition.

Let’s examine a few of the reasons that we heard. First, the rate of new requirements has been accelerating for many years, and it continues to accelerate even now. Architects today are faced with a nonstop and ever-increasing stream of feature requests and bugs to fix, driven by customer and business needs and by competitive pressures. If architects aren’t paying attention to the modularity of their system (and, no, microservices are not a panacea here), that system will quickly become an anchor—hard to understand, change, debug, and modify, and weighing down the business.

Second, while the level of abstraction in systems is increasing—we can and do regularly use many sophisticated services, blissfully unaware of how they are implemented—the complexity of the systems we are being asked to create is increasing at least as quickly. This is an arms race, and the architects aren’t winning! Architecture has always been about taming complexity, and that just isn’t going to go away anytime soon.

Speaking of raising the level of abstraction, model-based systems engineering (MBSE) has emerged as a potent force in the engineering field over the last decade or so. MBSE is the formalized application of modeling to support (among other things) system design. The International Council on Systems Engineering (INCOSE) ranks MBSE as one of a select set of “transformational enablers” that underlie the entire discipline of systems engineering. A model is a graphical, mathematical, or physical representation of a concept or a construct that can be reasoned about. INCOSE is trying to move the engineering field from a document-based mentality to a model-based mentality, where structural models, behavioral models, performance models, and more are all used consistently to build systems better, faster, and cheaper. MBSE per se is beyond the scope of this book, but we can’t help but notice that what is being modeled is architecture. And who builds the models? Architects.

Third, the meteoric growth (and unprecedented levels of employee turnover) that characterizes the world of information systems means that no one understands everything in any real-world system. Just being smart and working hard aren't good enough.

Fourth, despite having tools that automate much of what we used to do ourselves—think about all of the orchestration, deployment, and management functions baked into Kubernetes, for example—we still need to understand the quality attribute properties of these systems that we depend upon, and we need to understand the emergent quality attribute properties when we combine systems together. Most quality attributes—performance, security, availability, safety, and so on—are susceptible to “weakest link” problems, and those weakest links may only emerge and bite us when we compose systems. Without a guiding hand to ward off disaster, the composition is very likely to fail. That guiding hand belongs to an architect, regardless of their title.

Given these considerations, we felt safe and secure that there was indeed a need for this book.

But was there a need for a fourth edition? Again (and this should be abundantly obvious), we concluded an emphatic “yes”! Much has changed in the computing landscape since the last edition was published. Some quality attributes that were not previously considered have risen to importance in the daily lives of many architects. As software continues to pervade all aspects of our society, *safety* considerations have become paramount for many systems; think about all of the ways that software controls the cars that we now drive. Likewise, *energy efficiency* is a quality that few architects considered a decade ago, but now must pay attention to, from massive data centers with unquenchable needs for energy to the small (even tiny) battery-operated mobile and IoT devices that surround us. Also, given that we are, more than ever, building systems by leveraging preexisting components, the quality attribute of *integrability* is consuming ever-increasing amounts of our attention.

Finally, we are building different kinds of systems, and building them in different ways than a decade ago. Systems these days are often built on top of virtualized resources that reside in a cloud, and they need to provide and depend on explicit interfaces. Also, they are increasingly mobile, with all of the opportunities and challenges that mobility brings. So, in this edition we have added chapters on virtualization, interfaces, mobility, and the cloud.

As you can see, we convinced ourselves. We hope that we have convinced you as well, and that you will find this fourth edition a useful addition to your (physical or electronic) bookshelf.

Register your copy of *Software Architecture in Practice, Fourth Edition*, on the InformIT site for convenient access to updates and/or corrections as they become available. To start the registration process, go to informit.com/register and log in or create an account. Enter the product ISBN (9780136886099) and click Submit. Look on the Registered Products tab for an Access Bonus Content link next to this product, and follow that link to access any available bonus materials. If you would like to be notified of exclusive offers on new editions and updates, please check the box to receive email from us.

Acknowledgments

We are profoundly grateful to all the people with whom we collaborated to produce this book.

First and foremost, we extend our gratitude to the co-authors of individual chapters. Their knowledge and insights in these areas were invaluable. Our thanks go to Cesare Pautasso of the Faculty of Informatics, University of Lugano; Yazid Hamdi of Siemens Mobile Systems; Greg Hartman of Google; Humberto Cervantes of Universidad Autonoma Metropolitana—Iztapalapa; and Yuanfang Cai of Drexel University. Thanks to Eduardo Miranda of Carnegie Mellon University’s Institute for Software Research, who wrote the sidebar on the Value of Information technique.

Good reviewers are essential to good work, and we are fortunate to have had John Hudak, Mario Benitez, Grace Lewis, Robert Nord, Dan Justice, and Krishna Guru lend their time and talents toward improving the material in this book. Thanks to James Ivers and Ipek Ozkaya for overseeing this book from the perspective of the SEI Series in Software Engineering.

Over the years, we have benefited from our discussions and writings with colleagues and we would like to explicitly acknowledge them. In particular, in addition to those already mentioned, our thanks go to David Garlan, Reed Little, Paulo Merson, Judith Stafford, Mark Klein, James Scott, Carlos Paradis, Phil Bianco, Jungwoo Ryoo, and Phil Laplante. Special thanks go to John Klein, who contributed one way or another to many of the chapters in this book.

In addition, we are grateful to everyone at Pearson for all their work and attention to detail in the countless steps involved in turning our words into the finished product that you are now reading. Thanks especially to Haze Humbert, who oversaw the whole process.

Finally, thanks to the many, many researchers, teachers, writers, and practitioners who have, over the years, worked to turn software architecture from a good idea into an engineering discipline. This book is for you.

This page intentionally left blank

1



What Is Software Architecture?

We are called to be architects of the future, not its victims.
—R. Buckminster Fuller

Writing (on our part) and reading (on your part) a book about software architecture, which distills the experience of many people, presupposes that

1. having a reasonable software architecture is important to the successful development of a software system and
2. there is a sufficient body of knowledge about software architecture to fill up a book.

There was a time when both of these assumptions needed justification. Early editions of this book tried to convince readers that both of these assumptions are true and, once you were convinced, supply you with basic knowledge so that you could apply the practice of architecture yourself. Today, there seems to be little controversy about either aim, and so this book is more about the supplying than the convincing.

The basic principle of software architecture is every software system is constructed to satisfy an organization's business goals, and that the architecture of a system is a bridge between those (often abstract) business goals and the final (concrete) resulting system. While the path from abstract goals to concrete systems can be complex, the good news is that software architectures can be designed, analyzed, and documented using known techniques that will support the achievement of these business goals. The complexity can be tamed, made tractable.

These, then, are the topics for this book: the design, analysis, and documentation of architectures. We will also examine the influences, principally in the form of business goals that lead to quality attribute requirements, that inform these activities.

In this chapter, we will focus on architecture strictly from a software *engineering* point of view. That is, we will explore the value that a software architecture brings to a development project. Later chapters will take business and organizational perspectives.

1.1 What Software Architecture Is and What It Isn't

There are many definitions of software architecture, easily discoverable with a web search, but the one we like is this:

The software architecture of a system is the set of structures needed to reason about the system. These structures comprise software elements, relations among them, and properties of both.

This definition stands in contrast to other definitions that talk about the system's "early" or "major" or "important" decisions. While it is true that many architectural decisions are made early, not all are—especially in Agile and spiral-development projects. It's also true that many decisions that are made early are not what we would consider architectural. Also, it's hard to look at a decision and tell whether it's "major." Sometimes only time will tell. And since deciding on an architecture is one of the architect's most important obligations, we need to know which decisions an architecture comprises.

Structures, by contrast, are fairly easy to identify in software, and they form a powerful tool for system design and analysis.

So, there we are: Architecture is about reasoning-enabling structures.

Let's look at some of the implications of our definition.

Architecture Is a Set of Software Structures

This is the first and most obvious implication of our definition. A structure is simply a set of elements held together by a relation. Software systems are composed of many structures, and no single structure can lay claim to being *the* architecture. Structures can be grouped into categories, and the categories themselves provide useful ways to think about the architecture. Architectural structures can be organized into three useful categories, which will play an important role in the design, documentation, and analysis of architectures:

1. Component-and-connector structures
2. Module structures
3. Allocation structures

We'll delve more into these types of structures in the next section.

Although software comprises an endless supply of structures, not all of them are architectural. For example, the set of lines of source code that contain the letter "z," ordered by increasing length from shortest to longest, is a software structure. But it's not a very interesting one, nor is it architectural. A structure is architectural if it supports reasoning about the system and the system's properties. The reasoning should be about an attribute of the system that is important to some stakeholder(s). These include properties such as the functionality achieved by the system, the system's ability to keep operating usefully in the face of faults or attempts to take it down, the ease or difficulty of making specific changes to the system, the system's

responsiveness to user requests, and many others. We will spend a great deal of time in this book exploring the relationship between architecture and *quality attributes* like these.

Thus the set of architectural structures is neither fixed nor limited. What is architectural depends on what is useful to reason about in your context for your system.

Architecture Is an Abstraction

Since architecture consists of structures, and structures consist of elements¹ and relations, it follows that an architecture comprises software elements and how those elements relate to each other. This means that architecture specifically and intentionally omits certain information about elements that is not useful for reasoning about the system. Thus an architecture is foremost an *abstraction* of a system that selects certain details and suppresses others. In all modern systems, elements interact with each other by means of interfaces that partition details about an element into public and private parts. Architecture is concerned with the public side of this division; private details of elements—details having to do solely with internal implementation—are not architectural. This abstraction is essential to taming the complexity of an architecture: We simply cannot, and do not want to, deal with all of the complexity all of the time. We want—and need—the understanding of a system’s architecture to be many orders of magnitude easier than understanding every detail about that system. You can’t keep every detail of a system of even modest size in your head; the point of architecture is to make it so you don’t have to.

Architecture versus Design

Architecture is design, but not all design is architecture. That is, many design decisions are left unbound by the architecture—it is, after all, an abstraction—and depend on the discretion and good judgment of downstream designers and even implementers.

Every Software System Has a Software Architecture

Every system has an architecture, because every system has elements and relations. However, it does not follow that the architecture is known to anyone. Perhaps all of the people who designed the system are long gone, the documentation has vanished (or was never produced), the source code has been lost (or was never delivered), and all we have at hand is the executing binary code. This reveals the difference between the architecture of a system and the *representation* of that architecture. Given that an architecture can exist independently of its description or specification, this raises the importance of *architecture documentation*, which is described in Chapter 22.

1. In this book, we use the term “element” when we mean either a module or a component, and don’t want to distinguish between the two.

Not All Architectures Are Good Architectures

Our definition is indifferent as to whether the architecture for a system is a good one or a bad one. An architecture may either support or hinder achieving the important requirements for a system. Assuming that we do not accept trial and error as the best way to choose an architecture for a system—that is, picking an architecture at random, building the system from it, and then hacking away and hoping for the best—this raises the importance of *architecture design*, which is treated in Chapter 20 and *architecture evaluation*, which will be dealt with in Chapter 21.

Architecture Includes Behavior

The behavior of each element is part of the architecture insofar as that behavior can help you reason about the system. The behavior of elements embodies how they interact with each other and with the environment. This is clearly part of our definition of architecture and will have an effect on the properties exhibited by the system, such as its runtime performance.

Some aspects of behavior are below the architect's level of concern. Nevertheless, to the extent that an element's behavior influences the acceptability of the system as a whole, this behavior must be considered part of the system's architectural design, and should be documented as such.

System and Enterprise Architectures

Two disciplines related to software architecture are system architecture and enterprise architecture. Both of these disciplines have broader concerns than software and affect software architecture through the establishment of constraints within which a software system, and its architect, must live.

System Architecture

A system's architecture is a representation of a system in which there is a mapping of functionality onto hardware and software components, a mapping of the software architecture onto the hardware architecture, and a concern for the human interaction with these components. That is, system architecture is concerned with the totality of hardware, software, and humans.

A system architecture will influence, for example, the functionality that is assigned to different processors and the types of networks that connect those processors. The software architecture will determine how this functionality is structured and how the software programs residing on the various processors interact.

A description of the software architecture, as it is mapped to hardware and networking components, allows reasoning about qualities such as performance and reliability. A description of the system architecture will allow reasoning about additional qualities such as power consumption, weight, and physical dimensions.

When designing a particular system, there is frequently negotiation between the system architect and the software architect over the distribution of functionality and, consequently, the constraints placed on the software architecture.

Enterprise Architecture

Enterprise architecture is a description of the structure and behavior of an organization's processes, information flow, personnel, and organizational subunits. An enterprise architecture need not include computerized information systems—clearly, organizations had architectures that fit the preceding definition prior to the advent of computers—but these days enterprise architectures for all but the smallest businesses are unthinkable without information system support. Thus a modern enterprise architecture is concerned with how software systems support the enterprise's business processes and goals. Typically included in this set of concerns is a process for deciding which systems with which functionality the enterprise should support.

An enterprise architecture will specify, for example, the data model that various systems use to interact. It will also specify rules for how the enterprise's systems interact with external systems.

Software is only one concern of enterprise architecture. How the software is used by humans to perform business processes and the standards that determine the computational environment are two other common concerns addressed by enterprise architecture.

Sometimes the software infrastructure that supports communication among systems and with the external world is considered a portion of the enterprise architecture; at other times, this infrastructure is considered one of the systems within an enterprise. (In either case, the architecture of that infrastructure is a *software* architecture!) These two views will result in different management structures and spheres of influence for the individuals concerned with the infrastructure.

Are These Disciplines in Scope for This Book? Yes! (Well, No.)

The system and the enterprise provide environments for, and constraints on, the software architecture. The software architecture must live within the system and the enterprise, and increasingly is the focus for achieving the organization's business goals. Enterprise and system architectures share a great deal with software architectures. All can be designed, evaluated, and documented; all answer to requirements; all are intended to satisfy stakeholders; all consist of structures, which in turn consist of elements and relationships; all have a repertoire of patterns at their respective architects' disposal; and the list goes on. So to the extent that these architectures share commonalities with software architecture, they are in the scope of this book. But like all technical disciplines, each has its own specialized vocabulary and techniques, and we won't cover those. Copious other sources exist that do.

1.2 Architectural Structures and Views

Because architectural structures are at the heart of our definition and treatment of software architecture, this section will explore these concepts in more depth. These concepts are dealt with in much greater depth in Chapter 22, where we discuss architecture documentation.

Architectural structures have counterparts in nature. For example, the neurologist, the orthopedist, the hematologist, and the dermatologist all have different views of the various

structures of a human body, as illustrated in Figure 1.1. Ophthalmologists, cardiologists, and podiatrists concentrate on specific subsystems. Kinesiologists and psychiatrists are concerned with different aspects of the entire arrangement's behavior. Although these views are pictured differently and have very different properties, all are inherently related and interconnected: Together they describe the architecture of the human body.

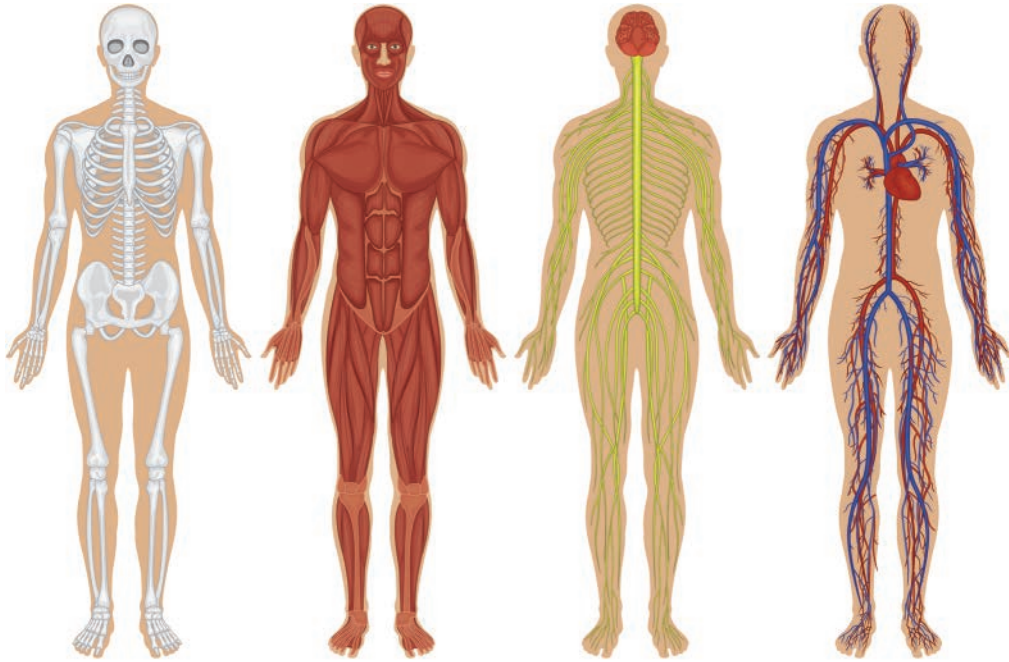


FIGURE 1.1 Physiological structures

Architectural structures also have counterparts in human endeavors. For example, electricians, plumbers, heating and air conditioning specialists, roofers, and framers are each concerned with different structures in a building. You can readily see the qualities that are the focus of each of these structures.

So it is with software.

Three Kinds of Structures

Architectural structures can be divided into three major categories, depending on the broad nature of the elements they show and the kinds of reasoning they support:

1. *Component-and-connector (C&C) structures* focus on the way the elements interact with each other at runtime to carry out the system's functions. They describe how the system is structured as a set of elements that have runtime behavior (components) and interactions (connectors). Components are the principal units of computation and could be services, peers, clients, servers, filters, or many other types of runtime element. Connectors are the communication vehicles among components, such as call-return, process synchronization operators, pipes, or others. C&C structures help answer questions such as the following:
 - What are the major executing components and how do they interact at runtime?
 - What are the major shared data stores?
 - Which parts of the system are replicated?
 - How does data progress through the system?
 - Which parts of the system can run in parallel?
 - Can the system's structure change as it executes and, if so, how?

By extension, these structures are crucially important for asking questions about the system's runtime properties, such as performance, security, availability, and more.

C&C structures are the most common ones that we see, but two other categories of structures are important and should not be overlooked.

Figure 1.2 shows a sketch of a C&C structure of a system using an informal notation that is explained in the figure's key. The system contains a shared repository that is accessed by servers and an administrative component. A set of client tellers can interact with the account servers and communicate among themselves using a publish-subscribe connector.

2. *Module structures* partition systems into implementation units, which in this book we call *modules*. Module structures show how a system is structured as a set of code or data units that have to be constructed or procured. Modules are assigned specific computational responsibilities and are the basis of work assignments for programming teams. In any module structure, the elements are modules of some kind (perhaps classes, packages, layers, or merely divisions of functionality, all of which are units of implementation). Modules represent a static way of considering the system. Modules are assigned areas of functional responsibility; there is less emphasis in these structures on how the resulting software manifests itself at runtime. Module implementations include packages, classes, and layers. Relations among modules in a module structure include uses, generalization (or "is-a"), and "is part of." Figures 1.3 and 1.4 show examples of module elements and relations, respectively, using the Unified Modeling Language (UML) notation.

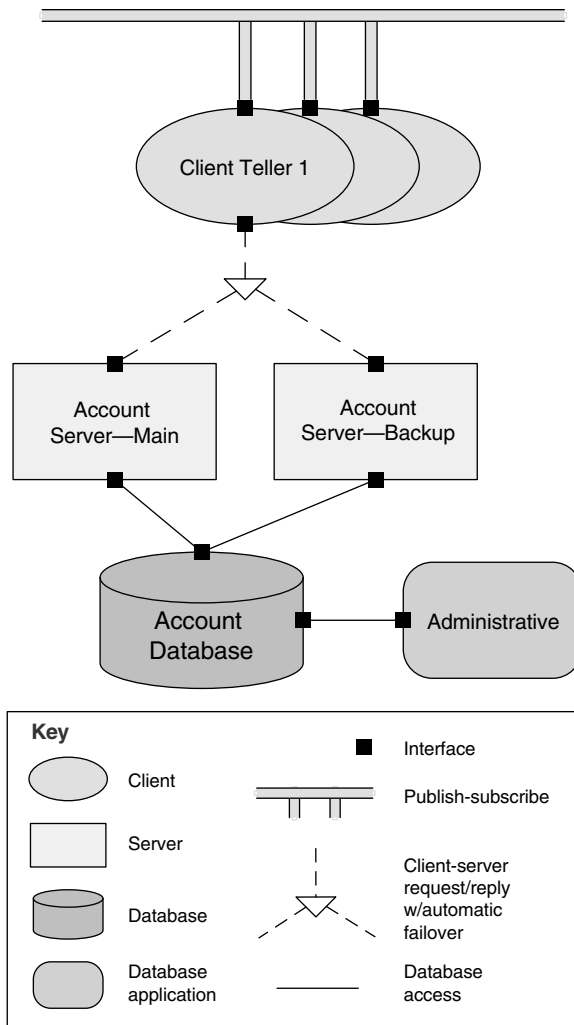


FIGURE 1.2 A component-and-connector structure

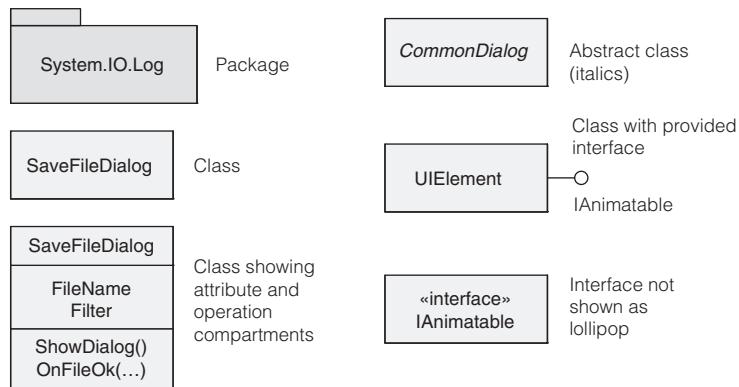


FIGURE 1.3 Module elements in UML

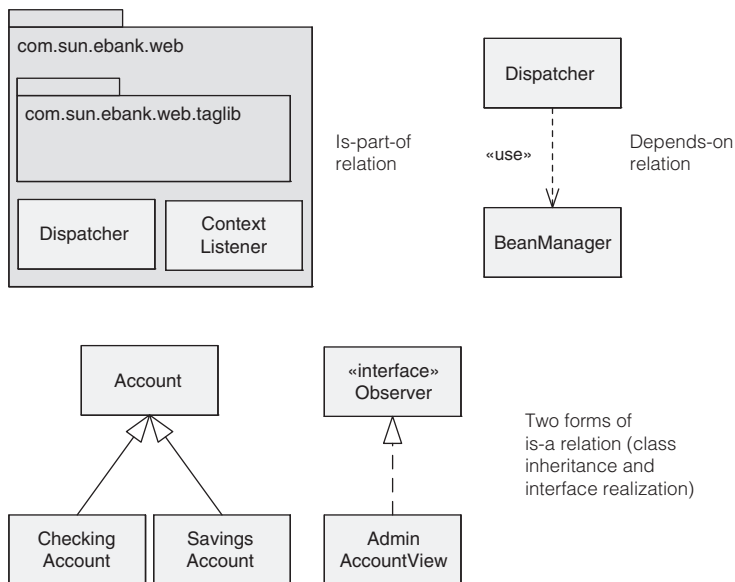


FIGURE 1.4 Module relations in UML

Module structures allow us to answer questions such as the following:

- What is the primary functional responsibility assigned to each module?
- What other software elements is a module allowed to use?
- What other software does it actually use and depend on?
- What modules are related to other modules by generalization or specialization (i.e., inheritance) relationships?

Module structures convey this information directly, but they can also be used to answer questions about the impact on the system when the responsibilities assigned to each module change. Thus module structures are the primary tools for reasoning about a system's modifiability.

3. *Allocation structures* establish the mapping from software structures to the system's nonsoftware structures, such as its organization, or its development, test, and execution environments. Allocation structures answer questions such as the following:

- Which processor(s) does each software element execute on?
- In which directories or files is each element stored during development, testing, and system building?
- What is the assignment of each software element to development teams?

Some Useful Module Structures

Useful module structures include:

- *Decomposition structure.* The units are modules that are related to each other by the “is-a-submodule-of” relation, showing how modules are decomposed into smaller modules recursively until the modules are small enough to be easily understood. Modules in this structure represent a common starting point for design, as the architect enumerates what the units of software will have to do and assigns each item to a module for subsequent (more detailed) design and eventual implementation. Modules often have products (such as interface specifications, code, and test plans) associated with them. The decomposition structure determines, to a large degree, the system's modifiability. That is, do changes fall within the purview of a few (preferably small) modules? This structure is often used as the basis for the development project's organization, including the structure of the documentation, and the project's integration and test plans. Figure 1.5 shows an example of a decomposition structure.
- *Uses structure.* In this important but often overlooked structure, the units are also modules, and perhaps classes. The units are related by the *uses* relation, a specialized form of dependency. One unit of software uses another if the correctness of the first requires the presence of a correctly functioning version (as opposed to a stub) of the second. The uses structure is used to engineer systems that can be extended to add functionality, or from which useful functional subsets can be extracted. The ability to easily create a subset of a system allows for incremental development. This structure is also the basis

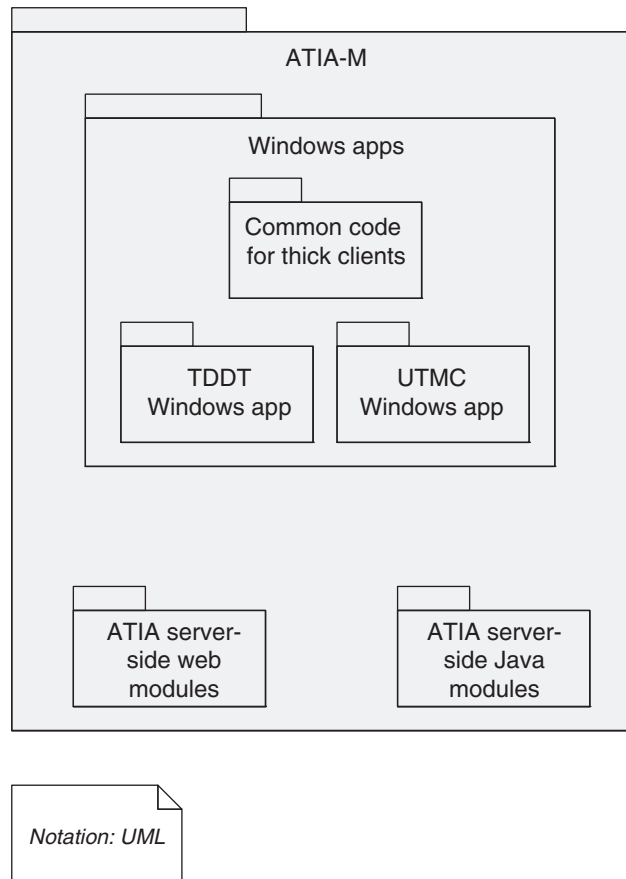


FIGURE 1.5 A decomposition structure

for measuring social debt—the amount of communication that actually is, as opposed to merely should be, taking place among teams—as it defines which teams should be talking to each other. Figure 1.6 shows a uses structure and highlights the modules that must be present in an increment if the module `admin.client` is present.

- **Layer structure.** The modules in this structure are called layers. A layer is an abstract “virtual machine” that provides a cohesive set of services through a managed interface. Layers are allowed to use other layers in a managed fashion; in strictly layered systems, a layer is only allowed to use a single other layer. This structure imbues a system with portability—that is, the ability to change the underlying virtual machine. Figure 1.7 shows a layer structure of the UNIX System V operating system.

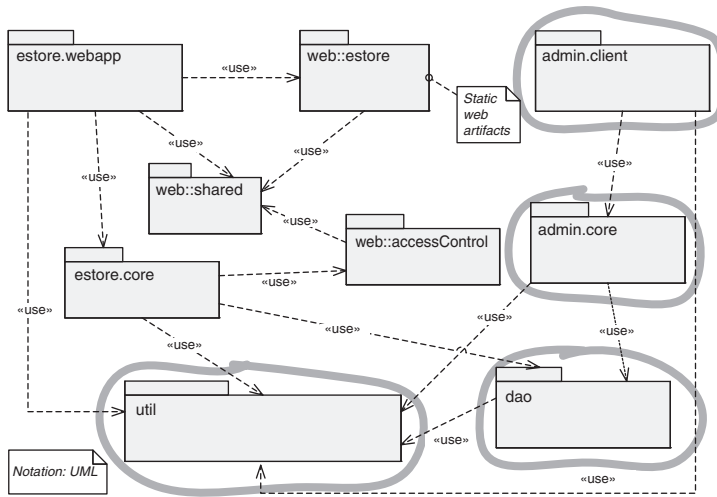


FIGURE 1.6 Uses structure

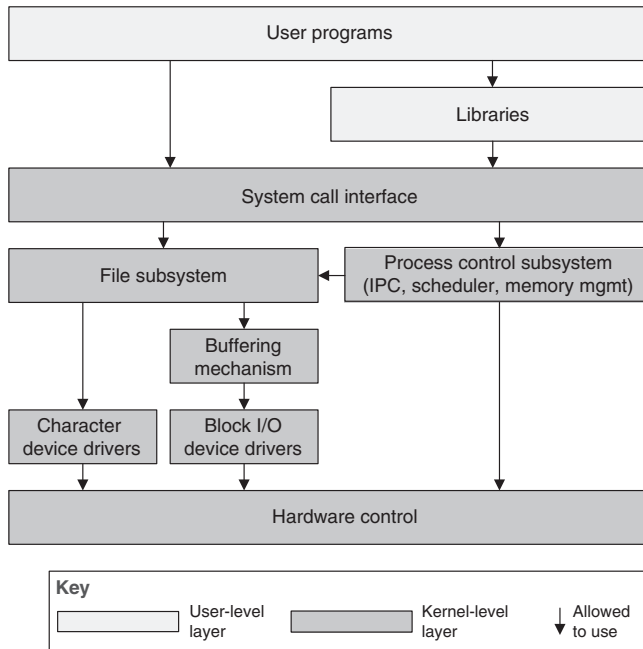


FIGURE 1.7 Layer structure

- *Class (or generalization) structure.* The modules in this structure are called classes, and they are related through an “inherits-from” or “is-an-instance-of” relation. This view supports reasoning about collections of similar behavior or capability and parameterized differences. The class structure allows one to reason about reuse and the incremental addition of functionality. If any documentation exists for a project that has followed an object-oriented analysis and design process, it is typically this structure. Figure 1.8 shows a generalization structure taken from an architectural expert tool.

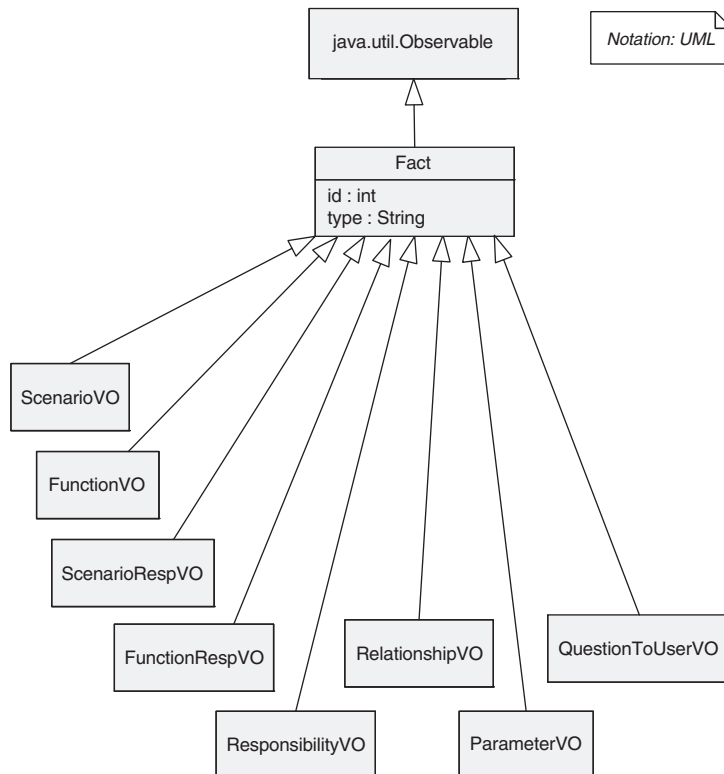


FIGURE 1.8 Generalization structure

- *Data model.* The data model describes the static information structure in terms of data entities and their relationships. For example, in a banking system, entities will typically include Account, Customer, and Loan. Account has several attributes, such as account number, type (savings or checking), status, and current balance. A relationship may dictate that one customer can have one or more accounts, and one account is associated with one or more customers. Figure 1.9 shows an example of a data model.

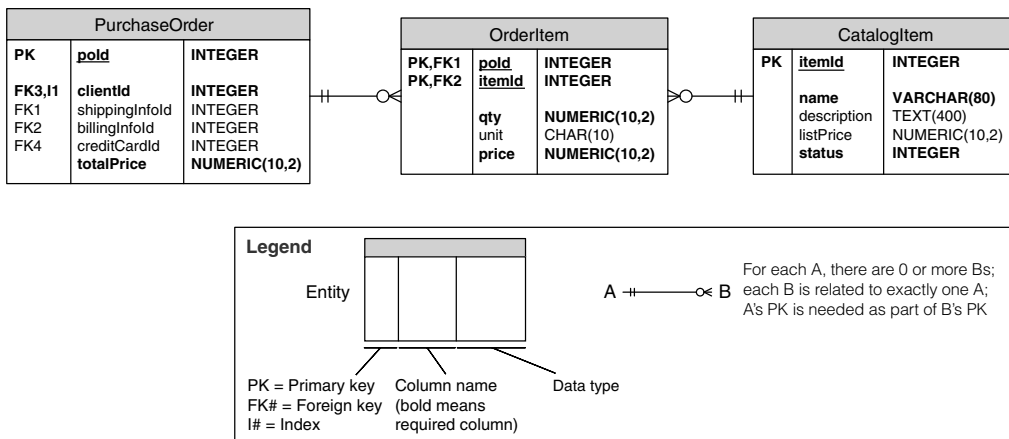


FIGURE 1.9 Data model

Some Useful C&C Structures

C&C structures show a runtime view of the system. In these structures, the modules just described have all been compiled into executable forms. Thus all C&C structures are orthogonal to the module-based structures and deal with the dynamic aspects of a running system. For example, one code unit (module) could be compiled into a single service that is replicated thousands of times in an execution environment. Or 1,000 modules can be compiled and linked together to produce a single runtime executable (component).

The relation in all C&C structures is *attachment*, showing how the components and the connectors are hooked together. (The connectors themselves can be familiar constructs such as “invokes.”) Useful C&C structures include:

- *Service structure.* The units here are services that interoperate through a service coordination mechanism, such as messages. The service structure is an important structure to help engineer a system composed of components that may have been developed independently of each other.
- *Concurrency structure.* This C&C structure allows the architect to determine opportunities for parallelism and the locations where resource contention may occur. The units are components, and the connectors are their communication mechanisms. The components are arranged into “logical threads.” A logical thread is a sequence of computations that could be allocated to a separate physical thread later in the design process. The concurrency structure is used early in the design process to identify and manage issues associated with concurrent execution.

Some Useful Allocation Structures

Allocation structures define how the elements from C&C or module structures map onto things that are not software—typically hardware (possibly virtualized), teams, and file systems. Useful allocation structures include:

- *Deployment structure.* The deployment structure shows how software is assigned to hardware processing and communication elements. The elements are software elements (usually a process from a C&C structure), hardware entities (processors), and communication pathways. Relations are “allocated-to,” showing on which physical units the software elements reside, and “migrates-to,” if the allocation is dynamic. This structure can be used to reason about performance, data integrity, security, and availability. It is of particular interest in distributed systems and is the key structure involved in the achievement of the quality attribute of deployability (see Chapter 5). Figure 1.10 shows a simple deployment structure in UML.

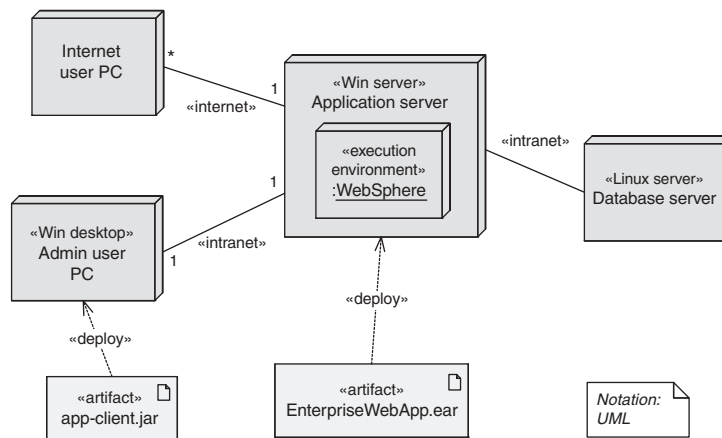


FIGURE 1.10 Deployment structure

- *Implementation structure.* This structure shows how software elements (usually modules) are mapped to the file structures in the system’s development, integration, test, or configuration control environments. This is critical for the management of development activities and build processes.
- *Work assignment structure.* This structure assigns responsibility for implementing and integrating the modules to the teams that will carry out these tasks. Having a work assignment structure be part of the architecture makes it clear that the decision about who does the work has architectural as well as management implications. The architect will know the expertise required on each team. Amazon’s decision to devote a single

team to each of its microservices, for example, is a statement about its work assignment structure. On large development projects, it is useful to identify units of functional commonality and assign those to a single team, rather than having them be implemented by everyone who needs them. This structure will also determine the major communication pathways among the teams: regular web conferences, wikis, email lists, and so forth.

Table 1.1 summarizes these structures. It lists the meaning of the elements and relations in each structure and tells what each might be used for.

Relating Structures to Each Other

Each of these structures provides a different perspective and design handle on a system, and each is valid and useful in its own right. Although the structures give different system perspectives, they are not independent. Elements of one structure will be related to elements of other structures, and we need to reason about these relations. For example, a module in a decomposition structure may be manifested as one, part of one, or several components in one of the C&C structures, reflecting its runtime alter-ego. In general, mappings between structures are many to many.

Figure 1.11 shows a simple example of how two structures might relate to each other. The image on the left shows a module decomposition view of a tiny client-server system. In this system, two modules must be implemented: the client software and the server software. The image on the right shows a C&C view of the same system. At runtime, ten clients are running and accessing the server. Thus this little system has two modules and eleven components (and ten connectors).

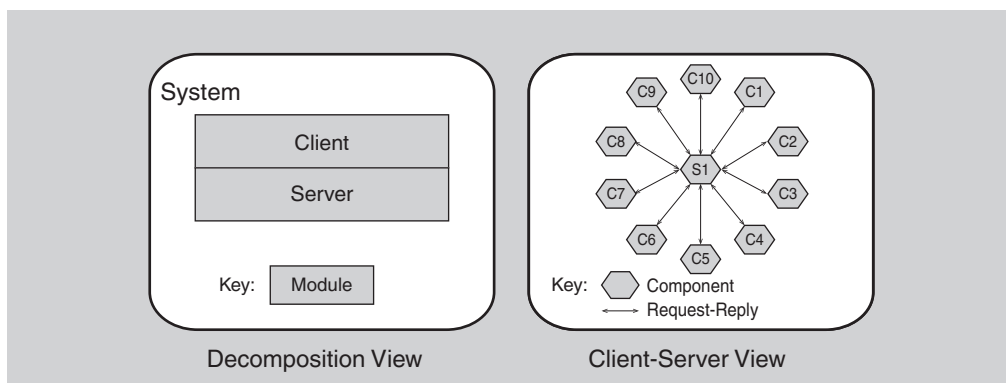


FIGURE 1.11 Two views of a client-server system

TABLE 1.1 Useful Architectural Structures

Software Structure		Element Types	Relations	Useful for	Quality Concerns Affected
Module structures	Decomposition	Module	Is a submodule of	Resource allocation and project structuring and planning; encapsulation	Modifiability
	Uses	Module	Uses (i.e., requires the correct presence of)	Designing subsets and extensions	"Subsetability," extensibility
	Layers	Layer	Allowed to use the services of; provides abstraction to	Incremental development; implementing systems on top of "virtual machines"	Portability, modifiability
	Class	Class, object	Is an instance of; is a generalization of	In object-oriented systems, factoring out commonality; planning extensions of functionality	Modifiability, extensibility
C&C structures	Data model	Data entity	{one, many}-to-{one, many}; generalizes; specializes	Engineering global data structures for consistency and performance	Modifiability, performance
	Service	Service, service registry	Attachment (via message-passing)	Scheduling analysis; performance analysis; robustness analysis	Interoperability, availability, modifiability
	Concurrency	Processes, threads	Attachment (via communication and synchronization mechanisms)	Identifying locations where resource contention exists, opportunities for parallelism	Performance
	Deployment	Components, hardware elements	Allocated to; migrates to	Mapping software elements to system elements	Performance, security, energy, availability, deployability
Allocation structures	Implementation	Modules, file structure	Stored in	Configuration control, integration, test activities	Development efficiency
	Work assignment	Modules, organizational units	Assigned to	Project management, best use of expertise and available resources, management of commonality	Development efficiency

Whereas the correspondence between the elements in the decomposition structure and the client-server structure is obvious, these two views are used for very different things. For example, the view on the right could be used for performance analysis, bottleneck prediction, and network traffic management, which would be extremely difficult or impossible to do with the view on the left. (In Chapter 9, we'll learn about the map-reduce pattern, in which copies of simple, identical functionality are distributed across hundreds or thousands of processing nodes—one module for the whole system, but one component per node.)

Individual projects sometimes consider one structure to be dominant and cast other structures, when possible, in terms of the dominant structure. Often, the dominant structure is the module decomposition structure, and for good reason: It tends to spawn the project structure, since it mirrors the team structure of development. In other projects, the dominant structure might be a C&C structure that shows how the system's functionality and/or critical quality attributes are achieved at runtime.

Fewer Is Better

Not all systems warrant consideration of many architectural structures. The larger the system, the more dramatic the difference between these structures tends to be; but for small systems, we can often get by with fewer structures. For example, instead of working with each of several C&C structures, usually a single one will do. If there is only one process, then the process structure collapses to a single node and need not be explicitly represented in the design. If no distribution will occur (that is, if the system is implemented on a single processor), then the deployment structure is trivial and need not be considered further. In general, you should design and document a structure only if doing so brings a positive return on the investment, usually in terms of decreased development or maintenance costs.

Which Structures to Choose?

We have briefly described a number of useful architectural structures, and many more are certainly possible. Which ones should an architect choose to work on? Which ones should the architect choose to document? Surely not all of them. A good answer is that you should think about how the various structures available to you provide insight and leverage into the system's most important quality attributes, and then choose the ones that will play the best role in delivering those attributes.

Architectural Patterns

In some cases, architectural elements are composed in ways that solve particular problems. These compositions have been found to be useful over time and over many different domains, so they have been documented and disseminated. These compositions of architectural elements, which provide packaged strategies for solving some of the problems facing a system, are called patterns. Architectural patterns are discussed in detail in Part II of this book.

1.3 What Makes a “Good” Architecture?

There is no such thing as an inherently good or bad architecture. Architectures are either more or less fit for some purpose. A three-tier layered service-oriented architecture may be just the ticket for a large enterprise’s web-based B2B system but completely wrong for an avionics application. An architecture carefully crafted to achieve high modifiability does not make sense for a throw-away prototype (and vice versa!). One of the messages of this book is that architectures can, in fact, be *evaluated*—one of the great benefits of paying attention to them—but such evaluation only makes sense in the context of specific stated goals.

Nevertheless, some rules of thumb should be followed when designing most architectures. Failure to apply any of these guidelines does not automatically mean that the architecture will be fatally flawed, but it should at least serve as a warning sign that should be investigated. These rules can be applied proactively for greenfield development, to help build the system “right.” Or they can be applied as analysis heuristics, to understand the potential problem areas in existing systems and to guide the direction of its evolution.

We divide our observations into two clusters: process recommendations and product (or structural) recommendations. Our process recommendations are as follows:

1. A software (or system) architecture should be the product of a single architect or a small group of architects with an identified technical leader. This approach is important to give the architecture its conceptual integrity and technical consistency. This recommendation holds for agile and open source projects as well as “traditional” ones. There should be a strong connection between the architects and the development team, to avoid “ivory tower,” impractical designs.
2. The architect (or architecture team) should, on an ongoing basis, base the architecture on a prioritized list of well-specified quality attribute requirements. These will inform the tradeoffs that always occur. Functionality matters less.
3. The architecture should be documented using *views*. (A view is simply a representation of one or more architectural structures.) The views should address the concerns of the most important stakeholders in support of the project timeline. This might mean minimal documentation at first, with the documentation then being elaborated later. Concerns usually are related to construction, analysis, and maintenance of the system, as well as education of new stakeholders.
4. The architecture should be evaluated for its ability to deliver the system’s important quality attributes. This should occur early in the life cycle, when it returns the most benefit, and repeated as appropriate, to ensure that changes to the architecture (or the environment for which it is intended) have not rendered the design obsolete.
5. The architecture should lend itself to incremental implementation, to avoid having to integrate everything at once (which almost never works) as well as to discover problems early. One way to do this is via the creation of a “skeletal” system in which the communication paths are exercised but which at first has minimal functionality. This skeletal system can be used to “grow” the system incrementally, refactoring as necessary.

Our structural rules of thumb are as follows:

1. The architecture should feature well-defined modules whose functional responsibilities are assigned on the principles of information hiding and separation of concerns. The information-hiding modules should encapsulate things likely to change, thereby insulating the software from the effects of those changes. Each module should have a well-defined interface that encapsulates or “hides” the changeable aspects from other software that uses its facilities. These interfaces should allow their respective development teams to work largely independently of each other.
2. Unless your requirements are unprecedented—possible, but unlikely—your quality attributes should be achieved by using well-known architectural patterns and tactics (described in Chapters 4 through 13) specific to each attribute.
3. The architecture should never depend on a particular version of a commercial product or tool. If it must, it should be structured so that changing to a different version is straightforward and inexpensive.
4. Modules that produce data should be separate from modules that consume data. This tends to increase modifiability because changes are frequently confined to either the production or the consumption side of data. If new data is added, both sides will have to change, but the separation allows for a staged (incremental) upgrade.
5. Don’t expect a one-to-one correspondence between modules and components. For example, in systems with concurrency, multiple instances of a component may be running in parallel, where each component is built from the same module. For systems with multiple threads of concurrency, each thread may use services from several components, each of which was built from a different module.
6. Every process should be written so that its assignment to a specific processor can be easily changed, perhaps even at runtime. This is a driving force in the increasing trends toward virtualization and cloud deployment, as we will discuss in Chapters 16 and 17.
7. The architecture should feature a small number of simple component interaction patterns. That is, the system should do the same things in the same way throughout. This practice will aid in understandability, reduce development time, increase reliability, and enhance modifiability.
8. The architecture should contain a specific (and small) set of resource contention areas, whose resolution is clearly specified and maintained. For example, if network utilization is an area of concern, the architect should produce (and enforce) for each development team guidelines that will result in acceptable levels of network traffic. If performance is a concern, the architect should produce (and enforce) time budgets.

1.4 Summary

The software architecture of a system is the set of structures needed to reason about the system. These structures comprise software elements, relations among them, and properties of both.

There are three categories of structures:

- Module structures show the system as a set of code or data units that have to be constructed or procured.
- Component-and-connector structures show the system as a set of elements that have runtime behavior (components) and interactions (connectors).
- Allocation structures show how elements from module and C&C structures relate to nonsoftware structures (such as CPUs, file systems, networks, and development teams).

Structures represent the primary engineering leverage points of an architecture. Each structure brings with it the power to manipulate one or more quality attributes. Collectively, structures represent a powerful approach for creating the architecture (and, later, for analyzing it and explaining it to its stakeholders). And, as we will see in Chapter 22, the structures that the architect has chosen as engineering leverage points are also the primary candidates to choose as the basis for architecture documentation.

Every system has a software architecture, but this architecture may or may not be documented and disseminated.

There is no such thing as an inherently good or bad architecture. Architectures are either more or less fit for some purpose.

1.5 For Further Reading

If you're keenly interested in software architecture as a field of study, you might be interested in reading some of the pioneering work. Most of it does not mention "software architecture" at all, as this phrase evolved only in the mid-1990s, so you'll have to read between the lines.

Edsger Dijkstra's 1968 paper on the T.H.E. operating system introduced the concept of layers [Dijkstra 68]. The early work of David Parnas laid many conceptual foundations, including information hiding [Parnas 72], program families [Parnas 76], the structures inherent in software systems [Parnas 74], and the uses structure to build subsets and supersets of systems [Parnas 79]. All of Parnas's papers can be found in the more easily accessible collection of his important papers [Hoffman 00]. Modern distributed systems owe their existence to the concept of cooperating sequential processes that (among others) Sir C. A. R. (Tony) Hoare was instrumental in conceptualizing and defining [Hoare 85].

In 1972, Dijkstra and Hoare, along with Ole-Johan Dahl, argued that programs should be decomposed into independent components with small and simple interfaces. They called their

approach structured programming, but arguably this was the debut of software architecture [Dijkstra 72].

Mary Shaw and David Garlan, together and separately, produced a major body of work that helped create the field of study we call software architecture. They established some of its fundamental principles and, among other things, catalogued a seminal family of architectural styles (a concept similar to patterns), several of which appear in this chapter as architectural structures. Start with [Garlan 95].

Software architectural patterns have been extensively catalogued in the series *Pattern-Oriented Software Architecture* [Buschmann 96 and others]. We also deal with architectural patterns throughout Part II of this book.

Early papers on architectural views as used in industrial development projects are [Soni 95] and [Kruchten 95]. The former grew into a book [Hofmeister 00] that presents a comprehensive picture of using views in development and analysis.

A number of books have focused on practical implementation issues associated with architectures, such as George Fairbanks' *Just Enough Software Architecture* [Fairbanks 10], Woods and Rozanski's *Software Systems Architecture* [Woods 11], and Martin's *Clean Architecture: A Craftsman's Guide to Software Structure and Design* [Martin 17].

1.6 Discussion Questions

1. Is there a different definition of software architecture that you are familiar with? If so, compare and contrast it with the definition given in this chapter. Many definitions include considerations like “rationale” (stating the reasons why the architecture is what it is) or how the architecture will evolve over time. Do you agree or disagree that these considerations should be part of the definition of software architecture?
2. Discuss how an architecture serves as a basis for analysis. What about decision making? What kinds of decision making does an architecture empower?
3. What is architecture's role in project risk reduction?
4. Find a commonly accepted definition of *system architecture* and discuss what it has in common with software architecture. Do the same for *enterprise architecture*.
5. Find a published example of a software architecture. Which structures are shown? Given its purpose, which structures *should* have been shown? What analysis does the architecture support? Critique it: What questions do you have that the representation does not answer?
6. Sailing ships have architectures, which means they have “structures” that lend themselves to reasoning about the ship's performance and other quality attributes. Look up the technical definitions for *barque*, *brig*, *cutter*, *frigate*, *ketch*, *schooner*, and *sloop*. Propose a useful set of “structures” for distinguishing and reasoning about ship architectures.

7. Aircraft have architectures that can be characterized by how they resolve some major design questions, such as engine location, wing location, landing gear layout, and more. For many decades, most jet aircraft designed for passenger transport have the following characteristics:

- Engines housed in nacelles slung underneath the wing (as opposed to engines built into the wings, or engines mounted on the rear of the fuselage)
- Wings that join the fuselage at the bottom (as opposed to the top or middle)

First, do an online search to find an example and a counter-example of this type of design from each of the following manufacturers: Boeing, Embraer, Tupolev, and Bombardier. Next, do some online research and answer the following question: What qualities important to aircraft does this design provide?

This page intentionally left blank

2



Why Is Software Architecture Important?

*Ah, to build, to build!
That is the noblest art of all the arts.*
—Henry Wadsworth Longfellow

If architecture is the answer, what was the question?

This chapter focuses on why architecture matters from a technical perspective. We will examine a baker's dozen of the most important reasons. You can use these reasons to motivate the creation of a new architecture, or the analysis and evolution of an existing system's architecture.

1. An architecture can either inhibit or enable a system's driving quality attributes.
2. The decisions made in an architecture allow you to reason about and manage change as the system evolves.
3. The analysis of an architecture enables early prediction of a system's qualities.
4. A documented architecture enhances communication among stakeholders.
5. The architecture is a carrier of the earliest, and hence most-fundamental, hardest-to-change design decisions.
6. An architecture defines a set of constraints on subsequent implementation.
7. The architecture dictates the structure of an organization, or vice versa.
8. An architecture can provide the basis for incremental development.
9. An architecture is the key artifact that allows the architect and the project manager to reason about cost and schedule.
10. An architecture can be created as a transferable, reusable model that forms the heart of a product line.
11. Architecture-based development focuses attention on the assembly of components, rather than simply on their creation.
12. By restricting design alternatives, architecture channels the creativity of developers, reducing design and system complexity.
13. An architecture can be the foundation for training of a new team member.

Even if you already believe us that architecture is important and don't need that point hammered home 13 more times, think of these 13 points (which form the outline for this chapter) as 13 useful ways to use architecture in a project, or to justify the resources devoted to architecture.

2.1 Inhibiting or Enabling a System's Quality Attributes

A system's ability to meet its desired (or required) quality attributes is substantially determined by its architecture. If you remember nothing else from this book, remember that.

This relationship is so important that we've devoted all of Part II of this book to expound on that message in detail. Until then, keep these examples in mind as a starting point:

- If your system requires high performance, then you need to pay attention to managing the time-based behavior of elements, their use of shared resources, and the frequency and volume of their interelement communication.
- If modifiability is important, then you need to pay attention to assigning responsibilities to elements and limiting the interactions (coupling) of those elements so that the majority of changes to the system will affect a small number of those elements. Ideally, each change will affect just a single element.
- If your system must be highly secure, then you need to manage and protect interelement communication and control which elements are allowed to access which information. You may also need to introduce specialized elements (such as an authorization mechanism) into the architecture to set up a strong "perimeter" to guard against intrusion.
- If you want your system to be safe and secure, you need to design in safeguards and recovery mechanisms.
- If you believe that scalability of performance will be important to the success of your system, then you need to localize the use of resources to facilitate the introduction of higher-capacity replacements, and you must avoid hard-coding in resource assumptions or limits.
- If your projects need the ability to deliver incremental subsets of the system, then you must manage intercomponent usage.
- If you want the elements from your system to be reusable in other systems, then you need to restrict interelement coupling so that when you extract an element, it does not come out with too many attachments to its current environment to be useful.

The strategies for these and other quality attributes are supremely architectural. But an architecture alone cannot guarantee the functionality or quality required of a system. Poor downstream design or implementation decisions can always undermine an adequate architectural design. As we like to say (*mostly* in jest): What the architecture giveth, the implementation may taketh away. Decisions at all stages of the life cycle—from architectural design to coding and implementation and testing—affect system quality. Therefore, quality is not completely a function of an architectural design. But that's where it starts.

2.2 Reasoning about and Managing Change

This is a corollary to the previous point.

Modifiability—the ease with which changes can be made to a system—is a quality attribute (and hence covered by the arguments in the previous section), but it is such an important quality that we have awarded it its own spot in the List of Thirteen. The software development community is coming to grips with the fact that roughly 80 percent of a typical software system’s total cost occurs *after* initial deployment. Most systems that people work on are in this phase. Many programmers and software designers *never* get to work on new development—they work under the constraints of the existing architecture and the existing body of code. Virtually all software systems change over their lifetimes, to accommodate new features, to adapt to new environments, to fix bugs, and so forth. But the reality is that these changes are often fraught with difficulty.

Every architecture, no matter what it is, partitions possible changes into three categories: local, nonlocal, and architectural.

- A local change can be accomplished by modifying a single element—for example, adding a new business rule to a pricing logic module.
- A nonlocal change requires multiple element modifications but leaves the underlying architectural approach intact—for example, adding a new business rule to a pricing logic module, then adding new fields to the database that this new business rule requires, and then revealing the results of applying the rule in the user interface.
- An architectural change affects the fundamental ways in which the elements interact with each other and will probably require changes all over the system—for example, changing a system from single-threaded to multi-threaded.

Obviously, local changes are the most desirable, so an *effective* architecture is one in which the most common changes are local, and hence easy to make. Nonlocal changes are not as desirable but do have the virtue that they can usually be staged—that is, rolled out—in an orderly manner over time. For example, you might first make changes to add a new pricing rule, then make the changes to actually deploy the new rule.

Deciding when changes are essential, determining which change paths have the least risk, assessing the consequences of proposed changes, and arbitrating sequences and priorities for requested changes all require broad insight into the relationships, performance, and behaviors of system software elements. These tasks are all part of the job description for an architect. Reasoning about the architecture and analyzing the architecture can provide the insights necessary to make decisions about anticipated changes. If you do not take this step, and if you do not pay attention to maintaining the conceptual integrity of your architecture, then you will almost certainly accumulate *architecture debt*. We deal with this subject in Chapter 23.